◇ Step 1: Problem Definition

The objective of the project was to build an **automatic License Plate Recognition (LPR) system** capable of:

- Detecting license plates from vehicle images
- Recognizing the alphanumeric text on the plate
- Producing accurate output under real-world conditions

The final system follows a two-stage OCR pipeline using PaddleOCR.

◇ Step 2: Environment Setup

You first prepared the development environment:

- Installed **PaddlePaddle (GPU version)**
- Installed PaddleOCR
- Verified CUDA and GPU availability
- Installed required Python dependencies

This ensured efficient training and faster inference.

◇ Step 3: Custom Dataset Preparation

You prepared a custom dataset specifically for license plates.

Detection Dataset:

- Vehicle images
- Annotated bounding boxes around license plates

Recognition Dataset:

- Cropped plate images
- Corresponding text labels

Example:

```
plate_001.jpg → UP32AB1234
```

You ensured dataset diversity:

- Different lighting conditions
- Motion blur
- Different font styles
- Slight rotation and perspective variation

This improves generalization.

◇ Step 4: Using Pretrained Models

Instead of training from scratch, you used **pretrained PaddleOCR models**.

Why?

- Faster convergence
- Better baseline accuracy
- Reduced training time

You loaded:

- Pretrained DB detection model
- Pretrained CRNN recognition model

Then fine-tuned them on your custom license plate dataset.

◇ Step 5: Detection Model Fine-Tuning (DB Model)

The detection stage uses the DB (Differentiable Binarization) model.

What You Did:

1. Configured dataset paths
2. Set training parameters (batch size, learning rate, epochs)
3. Loaded pretrained detection weights
4. Started fine-tuning
5. Saved best performing weights

What the Model Learned:

- Locate license plate region
- Distinguish plate from background

- Generate probability map
- Apply adaptive threshold map
- Output bounding boxes

This stage outputs cropped plate regions.

◇ Step 6: Recognition Model Fine-Tuning (CRNN Model)

Recognition stage uses CRNN architecture.

CRNN = CNN + BiLSTM + CTC

What You Did:

1. Prepared recognition dataset (image + text labels)
2. Updated character dictionary (A-Z, 0-9)
3. Loaded pretrained recognition weights
4. Fine-tuned on license plate dataset
5. Saved trained model

How Recognition Works Internally

1 CNN – Feature Extraction

- Converts cropped plate image into feature maps
- Extracts strokes and character patterns
- Converts image width into sequence features

2 BiLSTM – Sequence Modeling

- Processes features left to right and right to left
- Learns character dependencies
- Improves structured text prediction

3 CTC – Decoding

- Removes blank tokens
- Collapses duplicate predictions
- Produces final clean text output

Example:

Raw prediction:

```
U _ P P 3 3 2 _ A B 1 1 2 3 4
```

Final:

```
UP32AB1234
```

◇ Step 7: Model Saving

After training:

- Saved detection model weights
- Saved recognition model weights
- Organized model directories properly

This allowed clean separation between training and inference.

◇ Step 8: Inference Pipeline Implementation

You created a separate inference notebook.

Inference Steps:

1. Load trained detection model
2. Load trained recognition model
3. Provide vehicle image as input
4. Run detection stage
5. Extract bounding box
6. Crop license plate region
7. Pass cropped image to recognition model
8. Generate final text output
9. Measure inference time

Complete pipeline:

```
Vehicle Image
   ↓
Detection Model (DB)
   ↓
Crop License Plate
   ↓
Recognition Model (CRNN)
```

```
    ↓
Final Plate Number
```

◇ Step 9: GPU Inference Optimization

You:

- Enabled GPU inference
- Checked device availability
- Measured inference latency
- Compared performance

This makes the system deployment-ready.

◇ Step 10: Testing and Evaluation

You tested:

- Multiple real images
- Different lighting conditions
- Slight blur and rotation
- Edge cases

You evaluated detection accuracy and recognition correctness.

◇ Step 11: Project Structure Organization

You maintained:

- Separate training notebook
- Separate inference notebook
- Separate model directories
- Clear dataset structure

This shows production-level thinking.

🎯 Final Summary of What You Did

In this project, you:

- ✔ Built a two-stage OCR system
- ✔ Used pretrained models
- ✔ Fine-tuned on custom license plate dataset
- ✔ Trained detection and recognition separately
- ✔ Implemented CRNN architecture
- ✔ Used CTC decoding
- ✔ Built full inference pipeline
- ✔ Enabled GPU acceleration
- ✔ Tested on real-world images